# Campus Lost and Found

Project Stage 2 Endterm

## Working prototype, data analysis and smart technologies

SE-2623 | Week 9 | Data freeze: 30 October 2026 | Technical update: 1 November 2026

**Mengdigerey Zhanzhol, Galitskiy Daniil, Bisengaliev Daulet**

The prototype combines a Flask application, a real SQLite database, private photographs, an administrator analytics page, an external-user timing recorder and a labelled smart-locker simulation. Students can publish, search, contact an author and close a notice. Administrators can moderate notices and inspect the data.

The delivered database contains 122 fictional notices, 20 demo accounts and 10 categories. Three reproducible queries and charts demonstrate analysis of that dataset. The original Stage 1 requirements and success targets are preserved.

### Current evidence status

| Available now | Awaiting validation |
|---|---|
| 72/72 automated checks (1 November 2026); 5/5 invalid-access checks; real database workflow; three charts; working software simulation. | Five external participants have not yet completed the two timed tasks. The three user-derived usability findings are therefore pending. |
| Windows portable runtime, Linux launchers, full backup and non-destructive restore check, presentation and defense guide. | Current browser screenshots and three cache-disabled Fast 4G Network runs are not yet captured. The Linux launchers have not been executed on Linux. |

This is a review and testing version. Parts A and C include current server evidence, but the complete current-version screenshot sequence is still pending. Older screenshots are identified by version and date. No external participant results, browser timings or university identity verification are invented.

Report structure: A Prototype and requirements · B Testing · C Data analysis · D IoT · References. The 9-slide presentation and Russian/English defense guide are separate files.

# A1 Student journey and persisted data

The student journey starts with a local university-domain account and ends with an item marked returned. The term local matters: the application accepts the required email suffix but does not verify access to that mailbox or connect to university SSO.

| Step | Action and database effect | Evidence |
|---|---|---|
| 1 | Sign in or register a local student account. Password is stored as a scrypt hash; role is fixed to student. | test_stage2_core.py; registration and login tests |
| 2 | Publish a lost/found notice. One JPG/PNG may be attached. SQLite stores the notice and a random private filename. | Server workflow step 1; F2/F3 tests |
| 3 | Search the title and select a category; open the matching record. | Server workflow step 2; test_app.py |
| 4 | Use Contact author. The mailto link addresses the owner stored in users. | Server workflow step 3; mailto destination assertion |
| 5 | Edit the owned notice, save, and reload; mark it returned after resolving the case. | Server workflow steps 4-5; SQL row checks |
| 6 | Read the saved returned state on the list/detail page. | Owner-return persistence tests; archived image A1-3 |

A fresh server integration check ran against an isolated copy of the shipped database. It created notice #123 with one PNG, found it through the search, verified its mailto destination, edited its description and persisted status returned. The real administrator then deleted that test record. The source dataset remained at 122 rows.

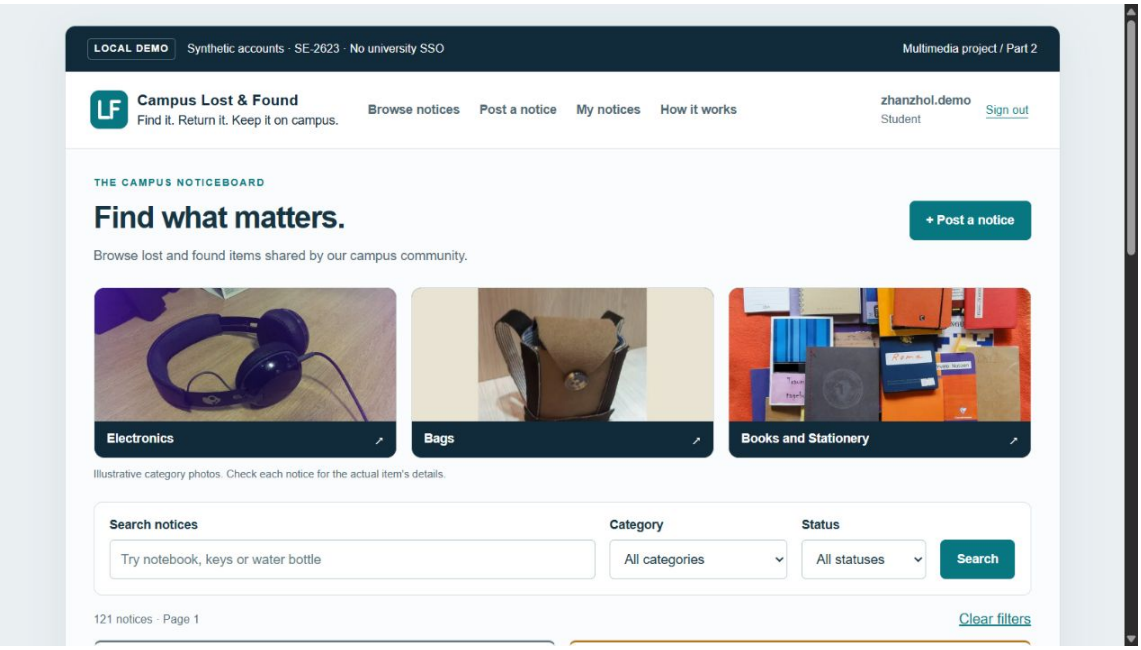
```
SELECT id, owner_id, title, status, photo_filename
FROM notices WHERE id = 123;
```

In that isolated check the stored owner_id was 1 and the final pre-removal status was returned. The associated filename was random, and the photo was served only through the authenticated route. The integrated request check verifies actual reads and writes; it is not a substitute for browser screenshots or external-user testing.

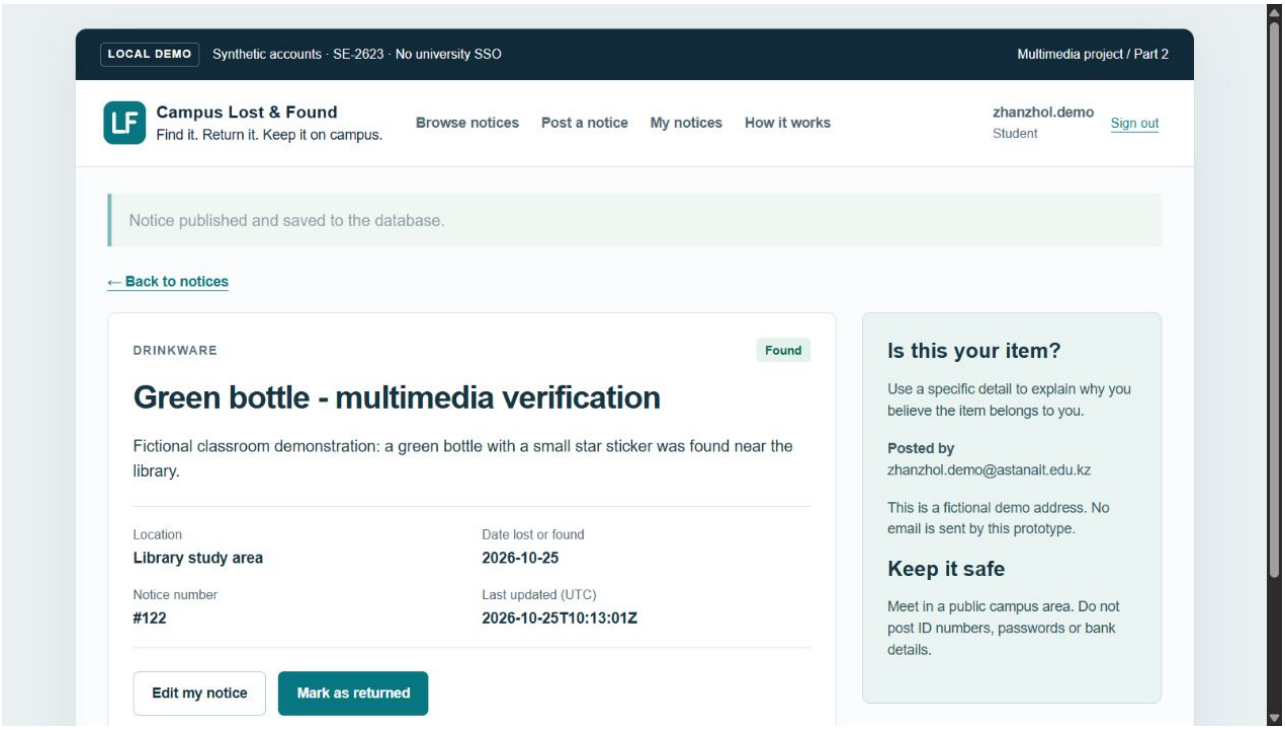
Evidence: Evidence/server_workflow_and_security.json. Full, current screenshots are still required for the submission version.

# A1 Archived visual context

These images are genuine screenshots captured for the preceding Assignment 4 build. They explain the retained notice workflow. They do not prove the new registration, upload, analytics or simulation interfaces.

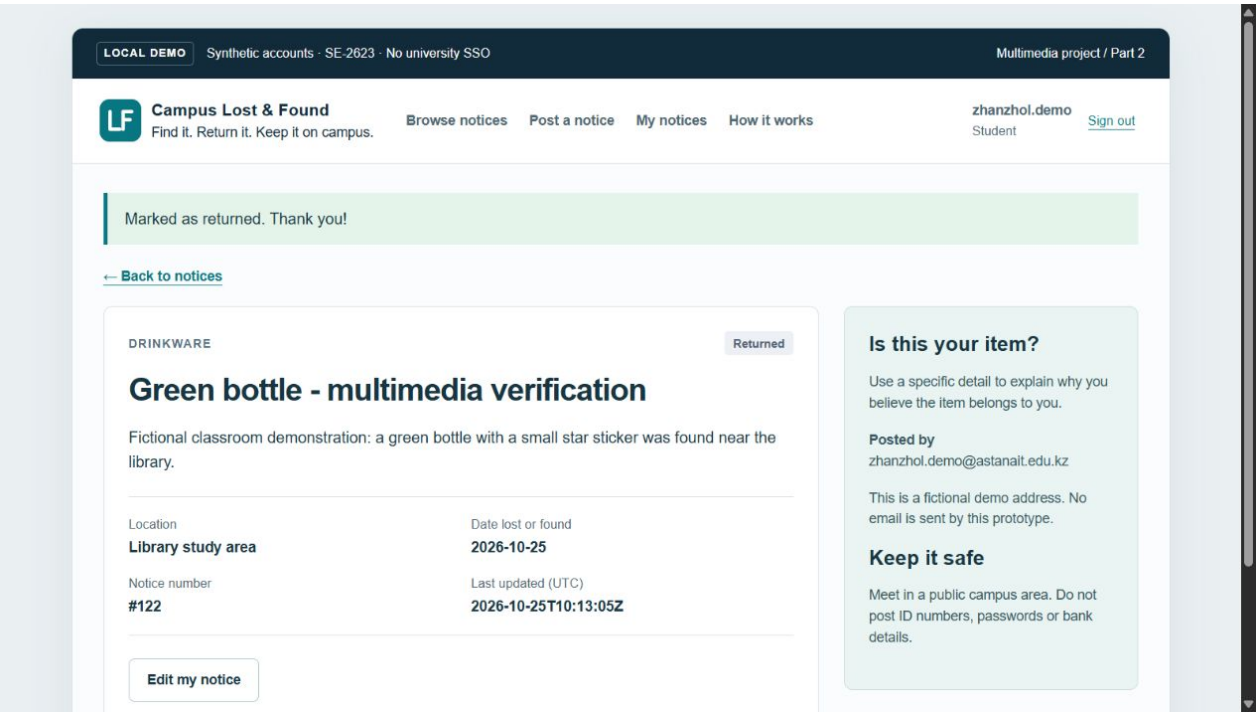



A1-1 Archive: the signed-in student opens the noticeboard and its search controls in the Assignment 4 build.




A1-2 Archive: a newly published fictional bottle notice is read back from SQLite.

# A1 Completion and administrator journey




A1-3 Archive: the owner marks the fictional item returned; the saved status is visible.

The administrator signs in as admin.demo@astanait.edu.kz, browses and opens the notice, selects Remove notice, reviews the confirmation page and confirms removal. The current server check verified that the notice row and its private photo disappear. A student attempting the same moderation route receives HTTP 403.

| Administrator step | Current check |
|---|---|
| Review notice details | Authenticated detail response returns the saved record. |
| Open removal confirmation | Administrator receives HTTP 200; ordinary student receives HTTP 403. |
| Confirm removal | CSRF-protected POST deletes the selected test row and its uploaded image. |
| Inspect analytics and extension | All three admin pages return HTTP 200; student access is separately tested. |

The complete current administrator screenshot sequence and a screenshot of the current analytics panel remain to be captured. The HTML/server checks and older student images above are explicitly different kinds of evidence.

# A2 Functional requirement status

IDs, wording and priorities below come from the final Stage 1 portrait report [5]. Status refers to the implemented local prototype; pending browser or external-service evidence is stated explicitly.

| ID / priority | Requirement | Status and evidence | Change since Stage 1 |
|---|---|---|---|
| F1 / Must | University account sign-in | Partly. Registration/domain/password tests. | Local sign-in added; university SSO and mailbox proof remain absent. |
| F2 / Must | Publish a lost or found notice | Done. Server workflow 1; create/persistence tests. | No target change; real SQLite writes replace the mockup. |
| F3 / Should | Attach one photo | Done. 11 core tests include PNG/JPEG limits, metadata and private access. | Implemented one photo up to 5 MiB; 12 MP safety limit added. |
| F4 / Must | Browse notices and open details | Done. Browse/detail tests and server workflow. | No change. |
| F5 / Must | Search title; filter by category | Done. Search/category tests and 100 HTTP searches. | No change. |
| F6 / Must | Contact via university email | Done. Saved owner mailto destination is verified. | No change. Opening the local mail client needs browser verification; no email was sent. |
| F7 / Must | Edit own notice; mark returned | Done. Edit/returned persistence and non-owner denial. | No change. |
| F8 / Must | Administrator removes misuse | Done. Admin removal passes; student receives 403. | No change; uploaded photo is removed with the notice. |

F3 uses a 5 × 1,024 × 1,024-byte file ceiling, the application’s interpretation of the original 5 MB limit. The whole multipart request is capped at 6 MiB. Valid images are re-encoded and reduced to at most 1600 pixels on a side; faces and personal identifiers should not be uploaded.

# A2 Nonfunctional requirement status

| ID / priority | Original requirement | Status and current evidence | Remaining scope |
|---|---|---|---|
| N1 / Must | Search p95 ≤ 2 seconds with 10 concurrent users. | Done in local HTTP test: p95 81.47 ms; 100 requests, 10 sessions. | Assets and painting excluded. Fast 4G page load is a separate pending measurement. |
| N2 / Must | Enforce university identity and owner/admin rights. | Partly. 5/5 selected access-denial checks pass; owner/admin enforcement works. | Email suffix does not prove university identity; approved identity integration is pending. |
| N3 / Must | Support 360 px screens and keyboard input. | Partly. Responsive styles, labels and focus treatment retained; earlier 360 px evidence archived. | New registration, analytics and locker pages need current keyboard and 360 px browser review. |
| N4 / Must | Pass 20/20 checks during a 30-minute demo. | Partly. 20/20 HTTP checks across 1800.0 seconds. | Browser offline/retry behavior is not included in the HTTP monitor. Rehearse and record it. |

All Must requirements are implemented or partly implemented. No target has been weakened. Incomplete identity verification, browser acceptance evidence and external-user testing remain explicit tasks rather than being counted as successes.

## What the availability log measures

The monitor uses real authenticated HTTP sessions and distributes 20 reads over 1,800 seconds: the main screen, title search, a saved notice, the publish form and analytics. It records UTC time, elapsed duration, status and response time. A successful response must contain the application content and must not be the sign-in form. It does not simulate a disconnected browser or measure user task success.

Evidence: Evidence/stability_30min.json and Evidence/search_performance.json.

# A3 Local execution and protection

## Windows

Extract the entire ZIP before running it. Open Campus_Lost_Found and double-click START.bat. Keep its terminal open and visit http://127.0.0.1:5055. A compatible Windows Python runtime and application dependencies are included. Run DATABASE.bat for the read-only SQLite inspector at http://127.0.0.1:8088.

## Linux and dual boot

Linux requires Bash, Python 3.10 or newer with venv/ensurepip, and Internet access for the first dependency setup. Open a terminal in Campus_Lost_Found and run the commands below. The launcher creates a separate Linux virtual environment in the user cache; it does not run the Windows interpreter or use Windows binary packages. Linux execution has not been verified on this Windows host.

```
bash START.sh
# Separate terminal, same folder:
bash DATABASE.sh
bash TESTS.sh
```

## Accounts for the classroom demonstration

Students: zhanzhol.demo@astanait.edu.kz, daniil.demo@astanait.edu.kz and daulet.demo@astanait.edu.kz. Administrator: admin.demo@astanait.edu.kz. The public demo password is CampusDemo!2026. These are synthetic records; never enter an actual university password.

## Security in the current version

Passwords use scrypt hashes. Every modifying request requires a CSRF token; parameterised queries and server-side owner/admin rules protect data operations. Login and registration attempts are rate limited. Images are validated, re-encoded without metadata, given random filenames and stored outside static files. Administrator-only analytics and research exports escape formula-like CSV cells.

The server binds to 127.0.0.1 and uses local HTTP. HTTPS, university SSO, mailbox verification and disk encryption are not configured or claimed. Local operating-system access can read SQLite and backup files regardless of application roles. A public deployment needs a persistent Python backend, storage, TLS and identity setup; this Flask package is not a standalone Netlify static site.

# A3 Backup and recovery evidence

BACKUP.bat or bash BACKUP.sh creates a full project data archive. It includes an online SQLite snapshot of campus.sqlite3, any research and telemetry databases, and private uploaded photographs. A SHA-256 manifest records each payload. The runtime session secret is excluded. Pause data-changing operations during a backup so the separate files form one consistent set.

RESTORE_CHECK.bat or bash RESTORE_CHECK.sh verifies the newest full archive and extracts it into a new scratch directory. It rejects unsafe paths, duplicate entries, unexpected payloads and incorrect hashes, then checks SQLite integrity and the photograph references. It does not replace the working database. The older backend-only backup command remains available for its original purpose.

| Check | Evidence |
| --- | --- |
| Snapshot plus uploaded files | test_backup_full.py verifies main, research, telemetry and photo payloads. |
| No destructive recovery | The working database remains unchanged during restore-check. |
| Archive safety | Traversal and corrupt-hash test cases are rejected. |
| Access enforcement example | A student opening the administrator removal action receives 403. |

The current five-case access check records 5/5 blocked attempts. HTTP 200 on the two invalid forms means an error page was rendered, not that an account was created or access granted. Session/database assertions verify the rejection.

| Check | Result |
| --- | --- |
| SC3-1 Register an address outside astanait.edu.kz | 200; blocked |
| SC3-2 Sign in with an incorrect password | 200; blocked |
| SC3-3 Anonymous visitor opens private notice details | 302; blocked |
| SC3-4 Student edits another owner notice | 403; blocked |
| SC3-5 Student opens administrator removal action | 403; blocked |

# A4 Week 10 plan and team contributions

No Midterm corrections were reported by the team. Exact calendar dates were not supplied, so deadlines below use the course milestones. The user study and browser evidence must be completed before the Week 9 submission, then any resulting fixes are carried into Week 10.

| Remaining item | Owner | Due |
|---|---|---|
| Run five external participants; retain original 120 s and 60 s targets; select the three most important observed problems. | Galitskiy Daniil | Before Week 9 submission |
| Capture the current student/admin journeys, analytics screen and three Fast 4G runs; review 360 px, keyboard and offline retry. | Mengdigerey Zhanzhol | Before Week 9 submission |
| Fix high-priority usability failures, rerun failed acceptance checks and update the evidence. | Galitskiy Daniil + Zhanzhol | Week 10 |
| Resolve university identity integration; add TLS before any network/public deployment. If unavailable, retain an explicit local demonstration boundary. | Mengdigerey Zhanzhol | Week 10 readiness review |
| Run the Linux launch/test/backup sequence on the dual-boot laptop. | Bisengaliev Daulet | Before Week 9 demonstration |
| Correct captions of the Assignment 4 fallback video; later physical locker work needs device authentication, replay/offline handling and retention enforcement. | Bisengaliev Daulet | Captions by Week 10; physical pilot beyond this checkpoint |

## Contribution table

| Member | Requirements and build material | Charts and presentation |
|---|---|---|
| Mengdigerey Zhanzhol | F1-F3, F6-F8 and N2: account, notice/photo workflow, access protection and backup demonstration. | Explain the product workflow and security. Slides 1-3. |
| Galitskiy Daniil | F4-F5, N1 and user-study evidence: browsing/search, performance method and three SQL analyses. | Charts 1-3, interpretation and live query. Slides 4-6. |
| Bisengaliev Daulet | N3-N4 evidence coordination, multimedia fallback and the smart-locker extension. | IoT sector research, four-layer diagram and simulation. Slides 7-9. |

# Technical review by five AI subagents

Technical update - 1 November 2026. Five separate AI subagents reviewed isolated application copies on Windows. Their assigned suites passed 65/65 baseline tests: 15 + 11 + 21 + 14 + 4. These counts describe automated tests, not human participants or usability results.

| AI ID | Assigned scope | What was actually checked | Passed |
|---|---|---|---|
| AI-01 | Student workflow and media | Login/logout, CSRF, owner/admin rules, create/edit/return persistence, literal search, static media delivery and alt-attribute presence. | 15/15 |
| AI-02 | Registration and private uploads | Domain/password rules, fixed student role, scrypt hashes, JPEG/PNG validation and re-encoding, photo replacement/deletion and login-gated access. | 11/11 |
| AI-03 | Database and analytics | 14 backend and 7 analytics tests; separate read-only reconciliation of all three query outputs with the frozen 122-notice database and stats.json. | 21/21 |
| AI-04 | Research recorder and simulated locker | Server timing and first-attempt deduplication, cohort/consent rules, CSV protection; simulated door warning, heartbeat and retained history. | 14/14 |
| AI-05 | Backup and recovery | Database/photo snapshots and hashes; scratch restore preserves live data; traversal/corruption rejection. Linux launchers inspected only. | 4/4 |

## Defects found and corrected

Additional input-boundary probes found SQLite integer overflows in numeric filters and notice-ID routes. Validation now rejects those inputs before database binding; four regression methods were added. Static backup review found a writer/restore manifest-size mismatch. Both now enforce the same 1 MiB bound before publication or extraction; three regression methods were added.

After both corrections, the complete Windows suite passed 72/72 tests in 30.069 seconds on 1 November 2026 (65 baseline + 7 new regression methods). The 30 October analysis data freeze remains unchanged. The historical baseline records remain separate from the fix records.

Packaged evidence: Evidence/Subagent_Review/{student,security,data,extensions,recovery}/result.json and their logs. Fix records: student/fix_result.json and recovery/fix_result.json in the same folder tree. Final run: Evidence/Update_2026-11-01/final-suite.log.

B1/B2 remain pending: five real external participants must still complete the two tasks, and three usability findings must come from their observations. AI checks do not replace that study. These reviews did not test browser usability, accessibility, Linux execution, university identity verification or physical locker hardware.

# B1 Original success criteria

Stage 1 defines a five-person threshold, so the brief's minimum of three external users is not sufficient for this project. Five volunteers outside the three-person team will each perform one first publish attempt and one first find attempt. The recorder starts empty and no team rehearsal or automated check is counted as a participant.

| Original criterion | Target | Method and measured result | Met? |
|---|---|---|---|
| 4/5 testers post a notice within 2 minutes | ≥4/5; ≤120 s | Five external people; anonymous code, device/browser and first task timing. No completed external sessions supplied yet. | Pending |
| 4/5 testers find an item within 1 minute | ≥4/5; ≤60 s | Same five people; search for the Red notebook in Books and Stationery. No completed external sessions supplied yet. | Pending |
| 5/5 invalid-access checks are blocked | 5 of 5 checks | Five named server checks on an isolated database copy: wrong domain, wrong password, anonymous details, foreign edit, student moderation. 5 blocked. | Yes 5/5 |

## Measurement procedure

The observer signs in as administrator in one browser window; the participant uses a separate private student window. The observer enters a code T01-T05, the actual device/browser and consent, then starts the server timer when the task card is presented. Stop immediately on completion or abandonment; enter outcome, any assistance and observations afterwards. Notes do not add to the elapsed task time.

The recorder keeps one first attempt per participant/task, caps the cohort at five people, retains failed attempts, and does not replace or retime completed records. Its unassisted count uses successful attempts within the original time limit. Results remain Pending until all ten attempts have final observations. Actual participant device data will come from those saved observations.

The administrator can download the observation CSV from /admin/research. The task cards and Russian observer guide are included in Testing. No personal names, email addresses or screen recordings of participants are needed.

# B2 Usability findings and B3 performance

## External-user findings

There are no completed external-user observations in this evidence freeze. Consequently, no three user-derived problems or Nielsen-heuristic rankings are reported yet. Code defects and team impressions are not substituted for those observations.

After the study, Daniil will rank actual problems by whether they prevent completion, increase task time or cause recoverable confusion. Each selected finding will identify its participant codes, observed action, violated heuristic and a specific change with its verification result. Nielsen's principles provide the classification: timely state feedback, error prevention, understandable language and recovery are relevant candidates, but a violation must be grounded in what people actually do [4].

## Search response benchmark

| Metric | Current observed value |
|---|---|
| Authenticated users / requests | 10 concurrent sessions / 100 requests |
| p95 / maximum | 81.47 ms / 88.213 ms |
| Original threshold | p95 ≤ 2,000 ms |
| Measured boundary | Loopback HTTP request until full HTML response; assets and painting excluded. |

Ten workers start together and each performs ten title searches against the running Waitress server. Login is outside the measured interval. The 95th percentile uses the nearest-rank position in the sorted 100 response times. This is evidence for the local search-response interpretation of N1, not a measurement of ten full mobile browser sessions.

## Required Fast 4G browser measurement

Three new Network-tab runs with Disable cache and the Fast 4G preset are pending. No current mean is claimed. The older Assignment 4 files contain a separate before/after media experiment, which is archived for provenance and is not substituted for this Endterm test.

The required capture procedure is: sign in, open the main screen, open Network, enable Disable cache, select Fast 4G, clear the log and reload three times. Record the Load value, transferred bytes, request count and a screenshot for each run; report the arithmetic mean of the three Load values. Keep the same viewport and machine, and distinguish this browser page-load metric from search p95.

# C1 and C2 Data and analytical questions

The main database is the Assignment 3 three-table model extended by the intervening live demonstrations: users, categories and notices. It contains 122 synthetic notices, exceeding the 100-record minimum. There are 96 open and 26 returned notices, 60 lost and 62 found. The freeze covers event dates 30 August to 25 October 2026.

| Question | Why an administrator would ask | Method |
|---|---|---|
| Q1 Which categories contain the open-notice backlog? | Find the categories that need review or follow-up. | Category LEFT JOIN; COUNT notices; conditional SUM open/returned. |
| Q2 Which recorded campus locations have most lost and found notices? | Compare location labels and prioritise location-data cleanup. | GROUP BY exact location; conditional counts for type. |
| Q3 How do lost and found events vary by calendar week? | See the event-time pattern and weeks with no recorded events. | Recursive week series, Monday-Sunday bins; LEFT JOIN and counts. |

SQL is stored in Analysis/01_category_backlog.sql, 02_location_types.sql and 03_weekly_events.sql. The same statements drive the administrator dashboard. Open DATABASE.bat (or bash DATABASE.sh), choose the SQL query interface and paste a statement. Alternatively use the read-only backend command below. The analysis CSVs and stats.json contain the exact frozen outputs.

```
.\DB.bat query "SELECT status, COUNT(*) FROM notices GROUP BY status;"
```

On Linux, use the interpreter printed by the launcher, or the included DATABASE.sh inspector. The optional Analysis/reproduce_analysis.py script exports all query results and recreates the charts after installing its separate plotting requirements. Plotting packages are not needed to run the web application.

The database SHA-256 and query hashes are recorded in Analysis/stats.json. Analysis/frozen_campus.sqlite3 is an identical read-only analysis copy, so later live demonstrations do not prevent reproducing these charts. No rows were inserted to improve the chart pattern. Tests use isolated databases, and external observations live in a separate research.sqlite3 file.

# C3 Chart 1 Category backlog

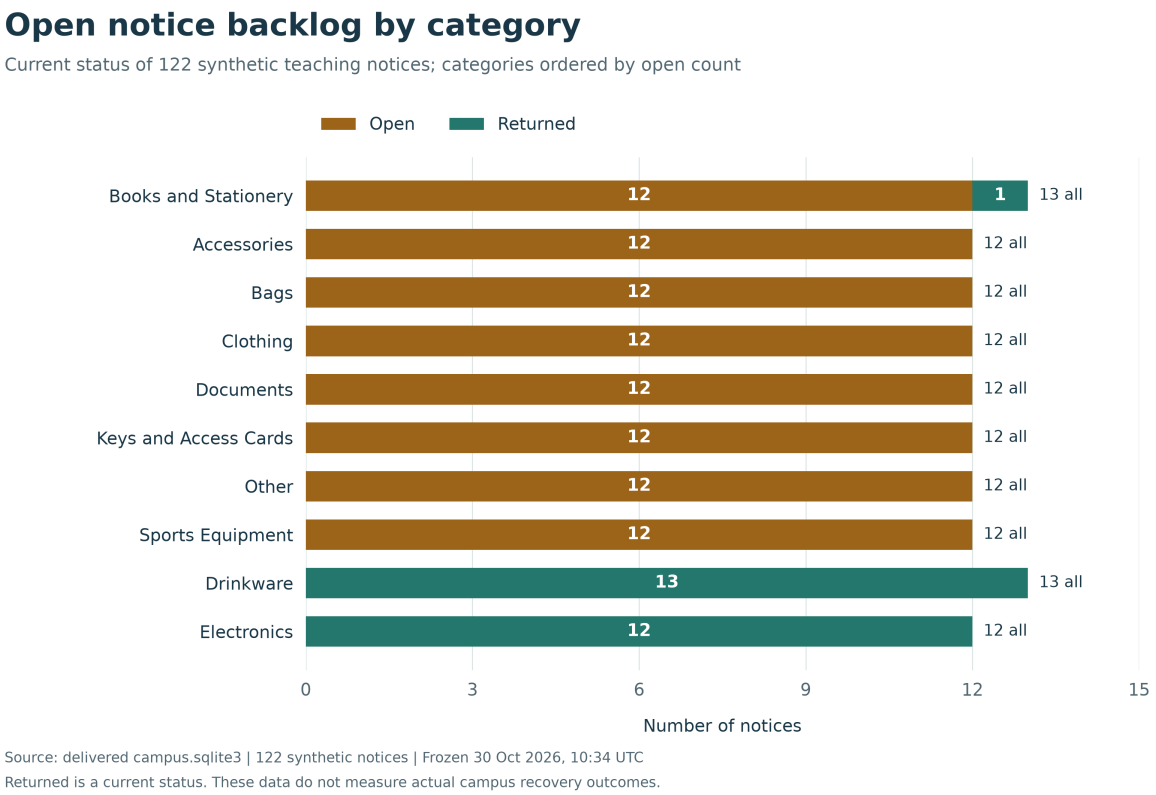



Figure C1. Open notices by category, with the full synthetic category totals retained in the accompanying CSV.

```
SELECT c.id AS category_id, c.name AS category,
       COUNT(n.id) AS total,
       SUM(CASE WHEN n.status = 'open' THEN 1 ELSE 0 END) AS open,
       SUM(CASE WHEN n.status = 'returned' THEN 1 ELSE 0 END) AS returned
FROM categories AS c
LEFT JOIN notices AS n ON n.category_id = c.id
GROUP BY c.id, c.name
ORDER BY open DESC, total DESC, c.name COLLATE NOCASE, c.id;
```

Eight categories each contain 12 open notices. Electronics and Drinkware contain zero open notices in this generated dataset. The LEFT JOIN preserves categories even when they have no notices; COUNT(n.id) avoids counting the artificial joined row as a notice.

# C3 Chart 2 Recorded locations

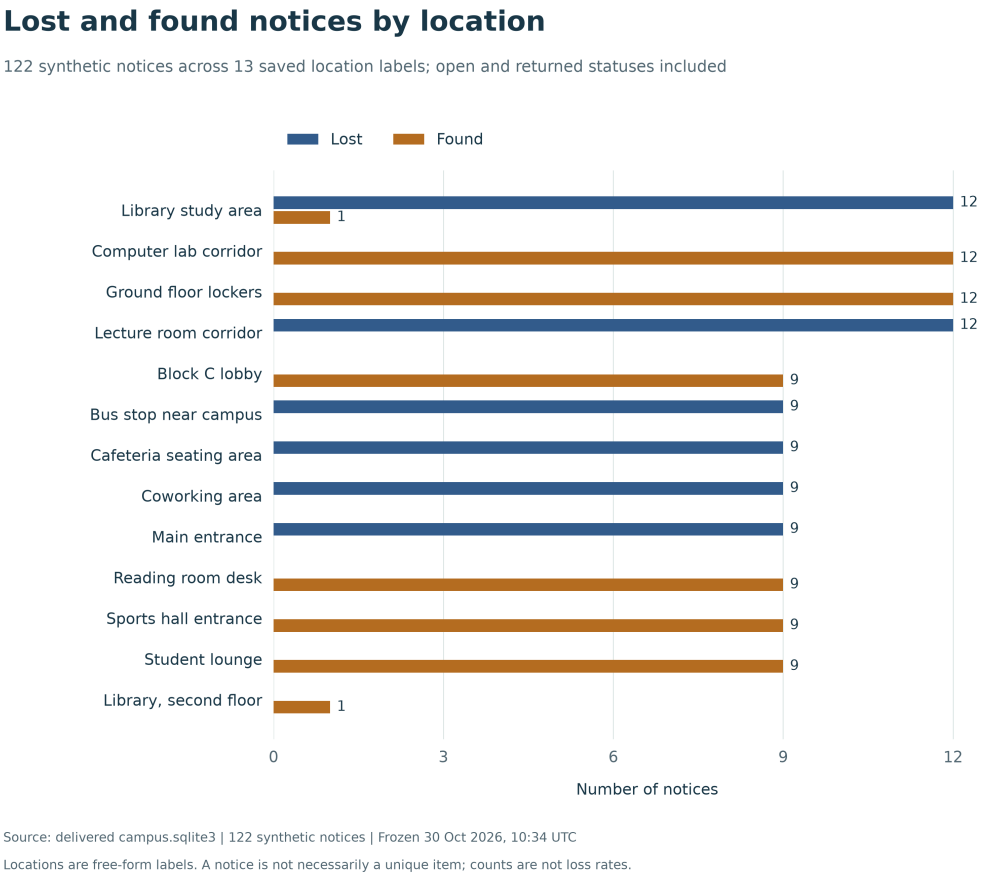



Figure C2. Counts of lost and found notices by exact location text; source is the frozen synthetic database.

```
SELECT location,
       SUM(CASE WHEN type = 'lost' THEN 1 ELSE 0 END) AS lost,
       SUM(CASE WHEN type = 'found' THEN 1 ELSE 0 END) AS found,
       COUNT(*) AS total
FROM notices
GROUP BY location
ORDER BY total DESC, location COLLATE NOCASE, location;
```

Library study area has 13 notices; Computer lab corridor, Ground floor lockers and Lecture room corridor have 12 each. The data model allows free text, so Library study area and Library, second floor are distinct groups. These counts are a query demonstration, not evidence of actual campus risk.

# C3 Chart 3 Weekly event pattern

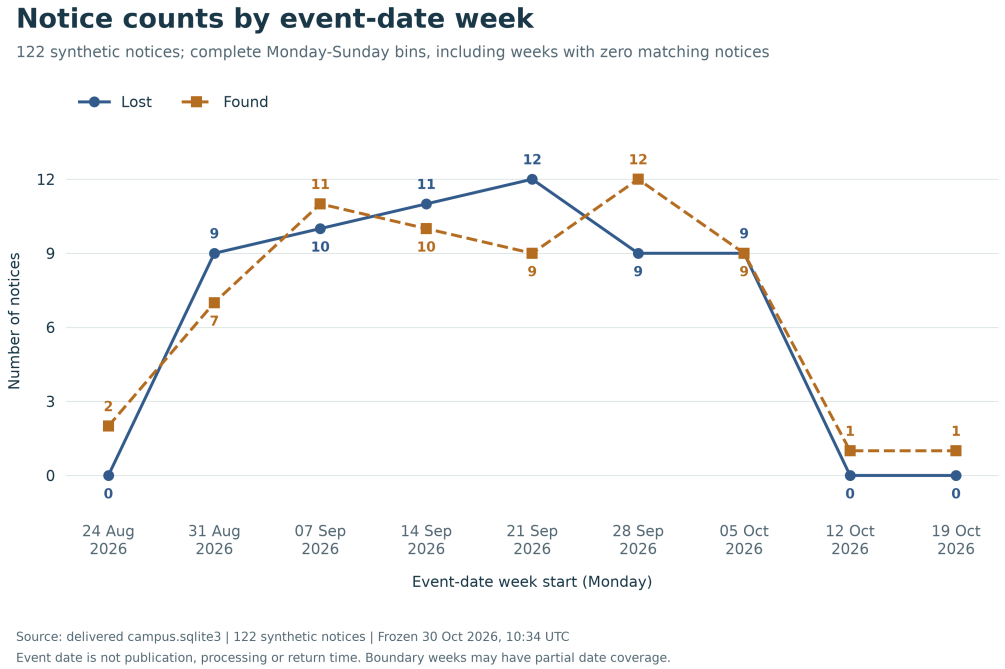



Figure C3. Monday-Sunday event-date weeks, including zero-count weeks. Dates describe when an item was lost/found, not when staff processed it.

```
WITH RECURSIVE
bounds AS (
    SELECT date(MIN(event_date), 'weekday 0', '-6 days') AS first_week,
           date(MAX(event_date), 'weekday 0', '-6 days') AS last_week
    FROM notices
),
weeks(week_start) AS (
    SELECT first_week FROM bounds WHERE first_week IS NOT NULL
    UNION ALL
    SELECT date(week_start, '+7 days') FROM weeks, bounds
    WHERE week_start < last_week
)
SELECT w.week_start,
       date(w.week_start, '+6 days') AS week_end,
       w.week_start || ' to ' || date(w.week_start, '+6 days') AS label,
       SUM(CASE WHEN n.type = 'lost' THEN 1 ELSE 0 END) AS lost,
       SUM(CASE WHEN n.type = 'found' THEN 1 ELSE 0 END) AS found,
       COUNT(n.id) AS total
FROM weeks AS w
LEFT JOIN notices AS n
  ON n.event_date BETWEEN w.week_start AND date(w.week_start, '+6 days')
GROUP BY w.week_start
ORDER BY w.week_start;
```

The recursive series makes missing weeks visible. The LEFT JOIN and COUNT(n.id) preserve zero counts. This dataset cannot answer average time to return: it has no dedicated, reliable returned_at timestamp.

# C4 Interpretation and C5 product integration

Interpretation · 175 words

The frozen dataset contains 122 synthetic notices: 96 open and 26 marked returned, comprising 60 lost and 62 found notices. Eight categories each have 12 open notices; Electronics and Drinkware have none. The generator strongly shapes this pattern: all 12 Electronics notices and all 13 Drinkware notices are marked returned. It cannot establish that these categories are easier to recover. Library study area has the largest location total, with 13 notices (12 lost and 1 found). Locations are free-form labels, so "Library study area" and "Library, second floor" remain separate groups. Normalising location choices would make future comparisons more consistent. Notice counts also do not identify unique physical items or account for how many people use each location. Weekly totals reach 21 in four consecutive September weeks. This reflects the seed schedule and later demonstration edits, not observed campus demand. Weeks are grouped by item event date, not publication, processing or return time; boundary weeks have partial date coverage. These aggregates verify the dashboard and suggest interface improvements, while real service evaluation requires independent observations.

## Data visible in the prototype

Sign in as admin.demo@astanait.edu.kz and open Analytics at /admin/analytics. The page reads the current SQLite database and displays the three aggregate series with accessible tables. Its summary cards show 122 notices, 96 open, 26 returned and 10 categories at this freeze. The underlying type aggregates total 60 lost and 62 found. A student is denied access. The aggregate CSV contains no emails or password hashes.

| Aggregate result | Frozen value | Where it comes from |
|---|---|---|
| All notices | 122 | COUNT of notices |
| Open / returned | 96 / 26 | Conditional status counts |
| Lost / found | 60 / 62 | Conditional type counts |
| Category / location / week groups | 10 / 13 / 9 | Three query outputs |

A current screenshot of the analytics page is pending. The current implementation is supported by seven analytics tests, independent SQL reconciliation and an authenticated HTTP 200 response. These records establish the data integration, but they do not replace the screenshot requested by C5.

A live defense query can change GROUP BY status to GROUP BY type. The values should change from 96/26 to 60/62 while both totals remain 122. This demonstrates that the chart is connected to stored records rather than hard-coded presentation values.

# D1 IoT in campus facilities

Real example · 175 words

Campus Lost & Found belongs to campus facilities within higher education. A real example is the University of Birmingham's partnership with Siemens: the university announced a rollout of 23,000 Enlighted IoT sensors, and Enlighted's case study describes installation in lighting fixtures [1,2]. Sensors collect energy consumption, occupancy, movement and temperature; Enlighted Manage analyses these readings to support decisions about room use and energy efficiency [2]. The manufacturer's documented architecture sends sensor data wirelessly over IEEE 802.15.4 to gateways, then over Ethernet/TCP/IP using TLS to Manage; this is a different wireless protocol from Wi-Fi [3]. Sensors also control lighting locally. Manage supports processing on a local server or in the cloud, although the campus sources do not identify which hosting option Birmingham uses [3]. The benefit is responsive lighting and better space planning, with campus data also supporting teaching and research [1,2]. One potential risk, in our assessment, is that detailed occupancy histories could expose people's routines if combined with other records. Restricting access, keeping aggregated reports and setting a retention limit would reduce that risk.

## Source and inference boundaries

The university announcement establishes the planned 23,000-sensor rollout [1]. The supplier case study describes the campus application and sensor measurements [2]. The manufacturer’s architecture explains IEEE 802.15.4, gateways and TLS to Manage [3]. The general vendor architecture is not evidence of Birmingham’s exact hosting configuration. Privacy risk from correlated occupancy histories is the team’s assessment.

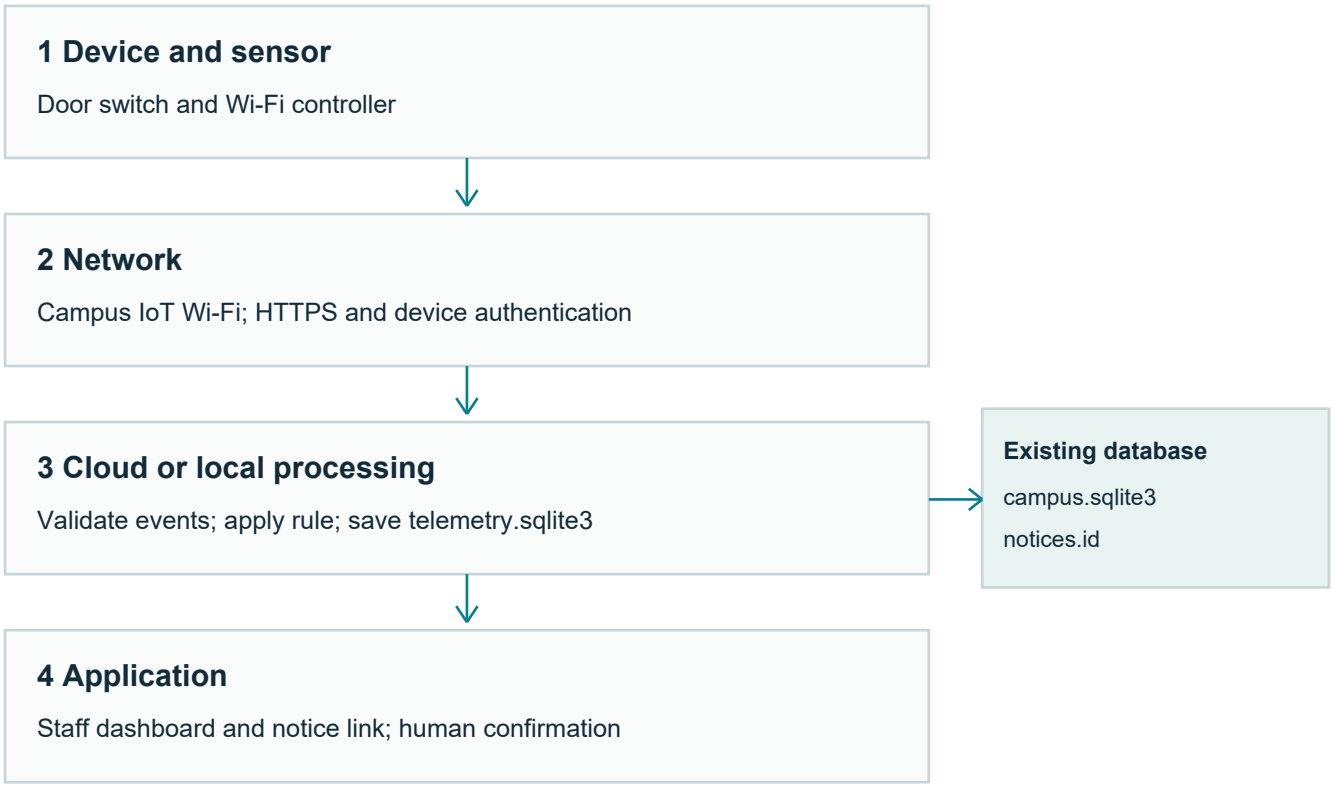
## Why it relates to this project

Both systems support campus facilities. Birmingham demonstrates sensing and analysis at building scale; Campus Lost and Found can apply the same device-to-application pattern to one monitored found-property locker. A door-state signal can help staff notice a cabinet left open, but it cannot prove which item is inside or who opened it.

The extension therefore preserves staff verification and existing notice ownership. It adds a small operational signal rather than automatically returning a notice or identifying a claimant. The classroom implementation is a software simulation, not connected campus hardware.

# D2 Proposed smart extension

The physical proposal monitors door state in a found-property locker. Processing may run on the same local server or an approved hosted service; the four-layer model does not require cloud hosting. The diagram describes the proposed physical deployment.




Processing validates notice_id against existing campus.sqlite3 notices.id.

The existing main database retains users, categories and notices. Separate telemetry storage records locker/device IDs, state, event type, timestamps, a sequence identifier, a server-validated notice reference and a source label. The notice relationship is validated by application logic; SQLite does not enforce a foreign key across these separate files.

A physical device would send each door-state change and a heartbeat every 60 seconds. A processing rule would evaluate every 5 seconds, warn after 30 seconds open and mark the device offline after 120 seconds without a reading. These physical-device, replay and offline features are proposals. The current demo uses a shorter 10-second rule evaluated on page refresh.

# D2 Data volume and privacy

| Design assumption | Value |
|---|---|
| Lockers / changes per day per locker | 10 / 20 |
| Heartbeats per locker per day | 1,440 at one per minute |
| Total readings per day | 10 × (1,440 + 20) = 14,600 |
| Payload allowance per reading | 200 bytes |
| Daily / 30-day payload | 2.92 MB / 87.6 MB, decimal units |

This is an illustrative capacity budget, not measured campus traffic or an actual SQLite file size. Database pages, indexes, transport headers, retries and backup copies add overhead. Twenty state changes correspond to about ten complete open/close cycles per locker per day.

The payload does not need a name, email address, university card number, GPS reading, photograph or audio recording. Nevertheless, the fixed locker location and exact time can become personal data when linked through a notice to its author. A door event does not establish who was physically present.

Assignment 3 already identifies location, date and author links as privacy-sensitive. The extension therefore uses staff-only routes, minimum data and a limited retention purpose. A physical rollout additionally needs per-device credentials, TLS, replay protection, IoT-network separation, operating-system access controls and suitable storage protection. These network controls are not claimed for the local HTTP demo.

The proposed policy retains raw telemetry for 30 days after its operational need and expires routine related backups under the seven-day backup policy. Deletion or anonymisation of unnecessary notice links and automatic expiry still need implementation. Application roles do not encrypt a copied database or backup.

# D3 Working software simulation

The administrator page /admin/smart-locker implements locker LF-01. Select a synthetic found notice, record an open state, wait at least ten seconds and refresh the page. The displayed warning is based on server timestamps. Record a closed state to resolve it. All readings are visibly labelled SIMULATED, and no notice is automatically marked returned.

| Implemented now | Physical pilot remains proposed |
| --- | --- |
| Local admin buttons and an optional local heartbeat script. | A reed switch and a microcontroller, provisioned device identity and network ingestion. |
| Separate telemetry.sqlite3; validated reference to a found notice. | Device sequence/replay validation, reliable offline buffering and reconciliation. |
| 10-second warning evaluated when the page is refreshed. | 30-second physical rule, periodic evaluation and 120-second offline detection. |
| Existing server-side admin/CSRF checks; retained event history. | TLS, device-to-locker authentication and automatic retention. |

The simulator only describes the last generated state. A missing closing reading could cause a false warning. The final build includes a safe way to close an obsolete assignment after its notice is removed or changed, retaining the telemetry history. Staff must still verify the item and claimant independently.

```
SELECT locker_id, notice_id, door_state,
       observed_at, source
FROM locker_events
ORDER BY id DESC LIMIT 6;
```

Run this read-only query against data/telemetry.sqlite3 after the demonstration. The optional simulate_locker.py command-line tool writes labelled local readings through the same validation logic; use its --help output for arguments. The simulator is optional in the brief and is included as a working software extension.

Defense sequence: explain the four physical layers, then explicitly switch to the simulation. Show an open event, continue the Week 10 discussion for ten seconds, refresh to show the warning, then close. State that the page refresh triggers evaluation and that no sensor hardware is connected.

# References and delivery record

[1] University of Birmingham. Partnership with Siemens. 8 September 2021. Open source.

[2] Enlighted. University of Birmingham case study. No printed date. Open source.

[3] Enlighted. Network and IT Design Guidance. Open source.

[4] Nielsen, J. Ten Usability Heuristics for User Interface Design. Reviewed 30 January 2024. Open source.

[5] SE-2623 team. Campus Lost and Found Stage 1 Midterm portrait report, final surname-corrected version. Functional and nonfunctional requirements, pages 3-4. Local source.

[6] Astana IT University ICT. Project Stage 2 End-term brief, Endterm_Week_9 (1).pdf, three pages. Local assignment source.

[7] SE-2623 team. Assignment 3 and Assignment 4 project artifacts, database, code, media-credit file and recorded demonstration. Local project sources.

Web sources checked 30 October 2026. Charts are original aggregations of synthetic project data. Archived screenshots are original captures of the Assignment 4 build. Media licences are preserved in Sources and static/media/CREDITS.txt.

## Delivered files and demonstration

The complete package includes Campus_Lost_Found source/runtime/database, Analysis queries/charts/CSVs, Evidence verification records, Testing task cards and instructions, this report, the nine-slide presentation and the Russian/English defense guide. The Assignment 4 MP4 is included as a fallback and is linked below; it is not the required live Endterm demonstration.

Assignment 4 fallback video on YouTube · Local MP4: 1920 × 1080, 30 fps, approximately 2:14. The existing uploaded captions need timing correction.

One team PDF is submitted to the LMS with all three names. Before that submission, complete five real external-user sessions, incorporate their three principal usability findings, capture the current journeys and analytics screen, and finish the specified three-run browser performance evidence. Each member must be able to reproduce the allocated live actions and answer individual questions.